Estratégia Concursos

Aula 18

DataPrev (Analista de TI - Desenvolvimento de Software) Passo Estratégico de Conhecimentos Específicos - 2024 (Pós-Edital)

Autor:

09 de Setembro de 2024

11225354498 - Maria Vitória Gonçalves Vieira

ARQUITETURA DE SOFTWARE

Sumário

Conteúdo 1

Glossário de termos 2

Roteiro de revisão 4

- Introdução e Conceitos Básicos 4
- Coesão e Acoplamento 6
- Arquitetura em Camadas 11
- Arquitetura MVC 19
- Arquitetura Distribuída 23
- Arquitetura de Microsserviços 24
- Arquitetura de Mainframe 27
- Arquitetura Peer-to-Peer (P2P) 29

Aposta estratégica 30

Questões Estratégicas 31

Questionário de revisão e aperfeiçoamento 40

- Perguntas 41
- Perguntas e Respostas 41

Lista de Questões Estratégicas 44

- Gabaritos 49

11225354498 - Maria Vitória Gonçalves Vieira

CONTEÚDO

Arquitetura de software. Conceitos iniciais, definições. Coesão e Acoplamento. Arquitetura em Camadas. Arquitetura Web. Modelo MVC. Arquitetura Distribuída. Arquitetura de Microsserviços. Arquitetura Mainframe. Arquitetura P2P.

ANÁLISE ESTATÍSTICA

Inicialmente, convém destacar o percentual de incidência do assunto, dentro da disciplina **Arquitetura de Software** em concursos/cargos similares. Quanto maior o percentual de cobrança de um dado assunto, maior sua importância.

Obs.: *um mesmo assunto pode ser classificado em mais de um tópico devido à multidisciplinaridade de conteúdo*.

Assunto	Relevância na disciplina em concursos similares
Web Services	**60%**
Padrões de Projeto	**20%**
Arquitetura e Software	**20%**

GLOSSÁRIO DE TERMOS

Faremos uma lista de termos que são relevantes ao entendimento do assunto desta aula. Caso tenha alguma dúvida durante a leitura, esta seção pode lhe ajudar a esclarecer.

Arquitetura de software: A estrutura organizacional de um sistema de software. Inclui componentes de software, as propriedades externamente visíveis desses componentes e as relações entre elas.

Coesão: A medida em que os componentes de um módulo de software estão relacionados entre si. Em uma arquitetura altamente coesa, cada módulo é responsável por uma tarefa específica.

Acoplamento: Descreve o grau de interdependência entre os módulos de software. Em um sistema com baixo acoplamento, os módulos são independentes uns dos outros.

Arquitetura em camadas: Um tipo de arquitetura de software que separa a funcionalidade do sistema em camadas, cada uma com funções específicas.

Arquitetura monolítica: Uma arquitetura de software em que a aplicação é construída como uma única unidade indivisível.

Arquitetura em duas camadas: Uma arquitetura em que a aplicação é dividida em duas partes principais: o cliente e o servidor.

Arquitetura em três camadas: Uma arquitetura que divide a aplicação em três componentes principais: a interface do usuário ou camada de apresentação, a lógica de negócio e a camada de acesso a dados.

Camada de apresentação: A camada que lida com a interação do usuário, incluindo a exibição de dados ao usuário e a captação de entrada do usuário.

Camada de lógica de negócio: A camada que lida com o processamento de dados e a aplicação de regras de negócio.

Camada de acesso a dados: A camada que lida com o armazenamento e recuperação de dados.

Arquitetura MVC: Um padrão de design que divide a aplicação em três componentes interconectados: Modelo, Visão e Controlador.

Modelo: Na arquitetura MVC, o modelo representa os dados e as regras de negócio da aplicação.

Visão: Na arquitetura MVC, a visão é responsável pela representação gráfica do modelo, ou seja, a interface com o usuário.

Controlador: Na arquitetura MVC, o controlador é o intermediário entre o modelo e a visão. Ele processa todas as solicitações de entrada e atualiza o modelo e a visão conforme necessário.

11225354498 - Maria Vitória Gonçalves Vieira

Arquitetura distribuída: Uma arquitetura em que os componentes do sistema são localizados em diferentes máquinas de rede.

Middleware: Software que atua como uma ponte entre diferentes aplicações, sistemas ou serviços, facilitando a comunicação e a gestão de dados.

Arquitetura de microsserviços: Um estilo de arquitetura que estrutura a aplicação como uma coleção de serviços leves, independentemente implantáveis e escaláveis.

Arquitetura mainframe: Uma arquitetura de computador centralizada em que uma grande máquina hospeda e executa todas as funções e serviços de processamento de dados.

Arquitetura P2P (Peer-to-Peer): Uma arquitetura de rede descentralizada em que todas as partes (peers) têm as mesmas responsabilidades e capacidades, compartilhando diretamente com cada

ROTEIRO DE REVISÃO

A ideia desta seção é apresentar um roteiro para que você realize uma revisão completa do assunto e, ao mesmo tempo, destacar aspectos do conteúdo que merecem atenção.

Introdução e Conceitos Básicos

A Arquitetura de Software pode ser definida como a **estrutura de alto nível de um sistema de software**. Ela representa a organização de um sistema em termos de **seus componentes, as interações entre esses componentes e os padrões que guiam tais interações**. Esses componentes podem ser variados, incluindo módulos de software, subsistemas, sistemas externos com os quais o software interage, e assim por diante.

A IEEE, ou Instituto de Engenheiros Eletricistas e Eletrônicos, fornece uma definição clara e amplamente aceita de arquitetura de software. De acordo com a IEEE Std 1471-2000 (agora ISO/IEC/IEEE 42010), a arquitetura de software é definida como:

11225354498 - Maria Vitória Gonçalves Vieira

> *"A organização fundamental de um sistema, incorporada em seus componentes, suas relações com cada um e com o ambiente, e os princípios que governam seu projeto e evolução."*

Esta definição enfatiza os aspectos estruturais e comportamentais da arquitetura de software, bem como o contexto no qual o sistema opera. Também destaca a importância dos princípios de design, que orientam as decisões tomadas durante o desenvolvimento do sistema, e a natureza evolutiva da arquitetura, que deve se adaptar à medida que os requisitos e as tecnologias mudam.

Importância da Arquitetura de Software e papel do Arquiteto de Software

A arquitetura do sistema é de suma importância na Engenharia de Software por várias razões.

Primeiramente, ela serve como um **plano para o sistema**. Ajuda a definir os componentes principais que compõem o sistema e como eles trabalham juntos. Além disso, fornece uma visão compartilhada e um entendimento comum entre as partes interessadas (stakeholders), incluindo desenvolvedores, gerentes de projeto e clientes.

Ainda, a arquitetura de software também auxilia nas decisões de design e implementação. Ela pode ajudar **a equilibrar as necessidades (requisitos funcionais) e restrições (requisitos não funcionais) do sistema**, como desempenho, segurança, manutenibilidade e outras propriedades não funcionais.

O **Arquiteto de Software** é o profissional responsável por definir a arquitetura do sistema. Eles precisam entender os requisitos do sistema e traduzi-los em uma arquitetura que pode ser implementada por desenvolvedores. O Arquiteto de precisa ter uma visão holística do sistema, considerando **tanto os aspectos técnicos quanto os de negócios**. Ele também deve ser capaz de comunicar efetivamente a arquitetura a várias partes interessadas.

Um exemplo bem conhecido de arquitetura de software é a arquitetura cliente-servidor, que é a base de muitas aplicações e sistemas de software modernos, particularmente na web.

Em uma arquitetura cliente-servidor, o sistema é dividido em duas partes principais:

1. **Servidor**: Esta é a entidade que possui e gerencia os recursos. Ele atende às solicitações dos clientes e executa funções, como processamento de dados, armazenamento e gerenciamento de banco de dados, entre outras tarefas.

11225354498 - Maria Vitória Gonçalves Vieira

2. **Cliente**: O cliente é a entidade que solicita recursos ou serviços do servidor. Ele pode ser um navegador web, uma aplicação de desktop, um aplicativo móvel, ou outro tipo de software que depende do servidor para realizar suas operações.

O Google é um exemplo emblemático de uso dessa arquitetura. Quando você realiza uma pesquisa no Google, seu navegador (o cliente) envia uma solicitação ao servidor do Google. O servidor processa a solicitação, procura os resultados relevantes e os envia de volta ao seu navegador.

Coesão e Acoplamento

Uma boa arquitetura de software, fundamentalmente, deve servir como uma ponte eficaz entre os requisitos do sistema e sua implementação. Ela deve acomodar funcionalidades funcionais e não funcionais, facilitar a manutenibilidade e a escalabilidade do sistema, promover a reusabilidade de componentes e permitir a interoperabilidade entre diferentes partes do sistema e com sistemas externos.

Em termos técnicos, uma boa arquitetura é constituída por **ALTA COESÃO e BAIXO ACOPLAMENTO.**

Acoplamento

Acoplamento é a medida do grau de **interdependência entre módulos ou componentes**. Refere-se a quão fortemente um componente ou módulo está ligado a outros. Um baixo acoplamento é preferível, pois módulos menos dependentes são mais fáceis de modificar, entender e usar em diferentes contextos. Eles também tendem a ser mais reutilizáveis, pois suas funcionalidades não estão fortemente entrelaçadas com as de outros módulos.

O baixo acoplamento em uma arquitetura de software traz várias vantagens, dentre as quais se destacam:

- **Manutenção mais fácil**: Com um acoplamento mais baixo, as mudanças em um módulo ou componente têm menos probabilidade de afetar outros módulos. Isso facilita a manutenção e a atualização do software, pois cada componente pode ser modificado sem a necessidade de alterar outros componentes.
- **Maior reusabilidade**: Componentes fracamente acoplados podem ser mais facilmente reutilizados em diferentes contextos ou em diferentes aplicações, pois suas funcionalidades não estão fortemente ligadas às de outros componentes.

11225354498 - Maria Vitória Gonçalves Vieira

- **Melhor escalabilidade**: Em uma arquitetura fracamente acoplada, como a arquitetura de microsserviços, cada componente pode ser escalado individualmente. Isso significa que se uma parte específica do sistema precisa de mais recursos (por exemplo, devido a um aumento no tráfego), apenas esse componente pode ser escalado, sem a necessidade de escalar todo o sistema.
- **Testabilidade aprimorada**: Em sistemas com baixo acoplamento, os componentes podem ser testados individualmente, o que torna o processo de teste mais eficiente e eficaz.
- **Maior resiliência**: Quando os componentes são fracamente acoplados, uma falha em um componente tem menos probabilidade de afetar outros componentes. Isso pode tornar todo o sistema mais resistente a falhas.

Um bom exemplo para ilustrar os conceitos de alto acoplamento e baixo acoplamento é comparar a arquitetura monolítica com a arquitetura de microsserviços.

Arquitetura Monolítica (Alto Acoplamento)

Um sistema monolítico é um sistema de software onde todos os componentes estão interligados e interdependentes. Em outras palavras, todas as funcionalidades são gerenciadas e servidas em um único programa, e qualquer alteração no sistema pode afetar todo o sistema.

Por exemplo, se você tivesse uma aplicação web monolítica para um e-commerce, todas as funcionalidades - incluindo gerenciamento de usuários, processamento de pedidos, interface do cliente, etc - estariam todos juntos em uma única aplicação. Se uma única parte do sistema falhar ou precisar ser atualizada, todo o sistema pode ser afetado, indicando um alto grau de acoplamento.

Arquitetura de Microsserviços (Baixo Acoplamento)

Por outro lado, a arquitetura de microsserviços representa um exemplo de baixo acoplamento. Neste modelo, cada funcionalidade do sistema é construída como um serviço separado, muitas vezes rodando em seu próprio processo e se comunicando com os outros por meio de mecanismos leves, como APIs RESTful.

No caso do exemplo do e-commerce, cada serviço - gerenciamento de usuários, processamento de pedidos, interface do cliente - seria um microsserviço separado. Cada um deles poderia ser

desenvolvido, implantado e escalado independentemente. Se um serviço falhar, ele pode ser reiniciado ou substituído sem afetar os outros serviços. Esta separação indica um baixo grau de acoplamento.

11225354498 - Maria Vitória Gonçalves Vieira

A literatura cita os seguintes tipos de acoplamento:

Tipo de Acoplamento	Descrição
Acoplamento por Conteúdo	**Ocorre quando um módulo faz uso de estruturas de dados ou de controle mantidas no escopo de outro módulo.**
Acoplamento Comum	**Ocorre quando um conjunto de módulos acessa uma área global de dados.**
Acoplamento por Controle	**Ocorre quando módulos passam decisões de controle a outros módulos, normalmente quando X passa uma "flag" para Y**
Acoplamento por "Carimbo" (Stamp)	**Ocorre quando X e Y se comunicam por parâmetros, sendo um deles, uma estrutura de dados**
Acoplamento por Dados	**Ocorre quando apenas uma lista de dados simples é passada como parâmetro de um módulo para o outro, com uma correspondência um-para-um de itens.**

Coesão

Coesão é o grau em que os elementos dentro de um módulo ou componente pertencem juntos. Em outras palavras, se refere à ideia de que cada parte de um sistema deve ter uma única **responsabilidade bem definida**. Um módulo coeso realiza uma única tarefa, reduzindo a complexidade e tornando o sistema mais fácil de entender, manter e expandir. Alta coesão é desejável em um sistema de software.

A alta coesão em uma arquitetura de software oferece várias vantagens significativas:

11225354498 - Maria Vitória Gonçalves Vieira

- **Facilidade de manutenção:** Quando um módulo ou componente tem alta coesão, ele é projetado para realizar uma tarefa ou conjunto de tarefas relacionadas. Isso significa que se uma alteração precisa ser feita na maneira como essa tarefa é realizada, as mudanças estarão confinadas a um módulo específico, tornando o processo de manutenção mais gerenciável.
- **Facilidade de compreensão:** Módulos coesos são mais fáceis de entender porque têm uma responsabilidade bem definida. Isso torna o código mais legível e a arquitetura do software mais fácil de entender, o que é particularmente útil para novos membros da equipe de desenvolvimento.
- **Reutilização de código:** Componentes com alta coesão são mais prováveis de serem reutilizáveis, pois realizam uma tarefa específica que pode ser necessária em outros lugares no software ou mesmo em outros projetos.
- **Confiabilidade:** A alta coesão pode aumentar a confiabilidade do software. Como cada módulo tem uma única responsabilidade, há menos chances de efeitos colaterais inesperados quando o módulo é modificado ou atualizado.
- **Testabilidade:** Módulos coesos são mais fáceis de testar, pois cada um tem uma responsabilidade bem definida. Isso pode resultar em casos de teste mais claros e diretos.
- **Paralelismo no desenvolvimento:** Com a alta coesão, diferentes equipes ou membros da equipe podem trabalhar em módulos diferentes simultaneamente sem se preocupar com a interferência nos trabalhos uns dos outros.

Para ilustrar os conceitos de alta e baixa coesão, vamos considerar o exemplo de uma aplicação de comércio eletrônico.

Arquitetura de Alta Coesão

Nesta arquitetura, cada módulo ou serviço teria uma única responsabilidade bem definida. Por exemplo, poderíamos ter módulos ou serviços separados para:

1. Gerenciamento de usuários: Este módulo seria responsável por todas as funções relacionadas ao gerenciamento de usuários, como criação de contas, autenticação, gerenciamento de perfil, etc.
2. Processamento de pedidos: Este módulo lidaria com todas as tarefas relacionadas ao processamento de pedidos, como aceitação de novos pedidos, atualização do status do pedido, notificação do cliente, etc.
3. Gestão de inventário: Este módulo seria responsável por gerenciar o inventário de produtos, incluindo adicionar novos produtos, atualizar a quantidade disponível de produtos, etc.

Cada um desses módulos ou serviços é altamente coeso, pois tem uma única responsabilidade bem definida.

11225354498 - Maria Vitória Gonçalves Vieira

Arquitetura de Baixa Coesão

Por outro lado, uma arquitetura de baixa coesão poderia ter módulos que combinam várias responsabilidades. Por exemplo, poderíamos ter um único módulo que lida com gerenciamento de usuários, processamento de pedidos e gerenciamento de inventário.

Este módulo não seria altamente coeso, pois combina várias responsabilidades diferentes. Como resultado, pode ser mais difícil de manter e testar, e as mudanças em um aspecto do módulo podem ter efeitos colaterais inesperados em outros aspectos.

A literatura cita os seguintes tipos de coesão:

Tipo de Coesão	Descrição
Coesão coincidental	**Deve ser evitada, ocorre quando um módulo realiza um conjunto de tarefas frouxamente relacionadas.**
Coesão Lógica	**Deve ser evitada, ocorre quando um módulo realiza um conjunto de tarefas que estão logicamente relacionadas, geralmente do mesmo tipo.**
Coesão Temporal	**Deve ser evitada, ocorre quando um módulo realiza tarefas que devem ser executadas no mesmo decurso de tempo, normalmente na inicialização ou término de funções.**
Coesão Procedural	**Ocorre quando os elementos de processamento do módulo são relacionadas e devem ocorrer em uma ordem específica.**
Coesão de Comunicação	**Ocorre quando todos os elementos de processamento do módulo se concentram em uma única área de uma estrutura de dados, geralmente porque usam o mesmo parâmetro.**

11225354498 - Maria Vitória Gonçalves Vieira

Coesão Sequencial	**Ocorre quando suas funções internas estão envolvidas em atividades de tal forma, que os dados de saída de uma atividade sirvam como dados de entrada para a próxima.**
Coesão Funcional	**Deve ser promovida, ocorre quando um módulo realiza uma única tarefa procedural distinta.**

Arquitetura em Camadas

A arquitetura em camadas, também conhecida como arquitetura de níveis ou arquitetura estratificada, é um estilo de arquitetura de software que **separa o sistema de software em um número de camadas, cada uma com uma responsabilidade específica**. Cada camada oferece serviços para a camada superior e atua como um cliente para a camada inferior.

Características:

- **Separação de preocupações:** Cada camada tem uma responsabilidade específica, o que ajuda na separação de preocupações. Por exemplo, na arquitetura de três camadas de uma aplicação web típica, a camada de apresentação lida com a interface do usuário, a camada de negócios lida com a lógica de negócios e a camada de dados lida com a persistência dos dados.
- **Independência:** As camadas são projetadas para serem independentes. Se uma camada é alterada, isso não deve afetar as outras camadas.
- **Reusabilidade:** Como cada camada é independente, elas podem ser reutilizadas em diferentes contextos.
- **Flexibilidade:** Como cada camada se concentra em uma única responsabilidade, é mais fácil modificar ou atualizar uma camada sem perturbar as outras.
- **Abstração:** Cada camada se comunica com as camadas adjacentes através de interfaces bem definidas, permitindo a abstração dos detalhes da implementação.

Um exemplo clássico de arquitetura em camadas, pertencente ao mundo de Engenharia de Redes, é a Arquitetura OSI/ISO. Esse modelo foi desenvolvido pela International Standards Organization (ISO) para facilitar a interconexão de sistemas abertos - isto é, sistemas que permitem qualquer par de sistemas a se comunicar, independentemente de suas tecnologias de rede subjacentes.

11225354498 - Maria Vitória Gonçalves Vieira

O Modelo OSI é composto por sete camadas, cada uma com suas próprias responsabilidades:

1. **Camada Física (Layer 1):** Trata da transmissão e recepção de dados brutos não estruturados em um meio físico. Ela define as características físicas dos dispositivos de rede, como as especificações elétricas dos cabos.
2. **Camada de Enlace de Dados (Layer 2):** Fornecer a transferência de dados entre entidades da rede e pode fornecer a detecção e correção de erros que podem ocorrer no nível físico.
3. **Camada de Rede (Layer 3):** Se responsabiliza pelo roteamento dos pacotes de dados, garantindo que eles se movam entre redes e cheguem ao destino correto.
4. **Camada de Transporte (Layer 4):** Oferece a transferência de dados confiável de ponta a ponta, fornecendo controle de fluxo, multiplexação, detecção de erros e correção.
5. **Camada de Sessão (Layer 5):** Controla as conexões entre computadores. Estabelece, gerencia e termina as sessões entre os aplicativos de comunicação.
6. **Camada de Apresentação (Layer 6):** Traduz, criptografa e comprime os dados. Também formatará os dados para serem enviados através da rede.
7. **Camada de Aplicação (Layer 7):** Fornece uma interface para o software se comunicar com a rede e inclui serviços como transferência de arquivos, compartilhamento de recursos, acesso remoto e serviços de e-mail.

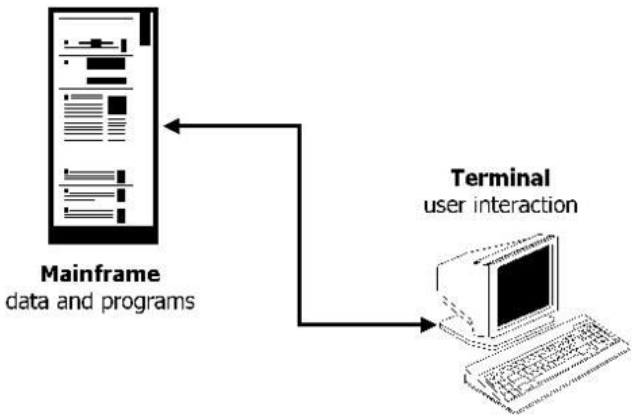
No mundo da Engenharia de Software, vamos falar um pouco sobre o histórico de arquitetura em camadas, desde a arquitetura monolítica (sem camadas) até as arquiteturas modernas.

Arquitetura Monolítica

Na arquitetura monolítica, todas as funcionalidades e componentes do software, como a interface do usuário, a lógica de negócios e o gerenciamento de banco de dados, estão contidos em um único programa. Esses componentes são interdependentes e interconectados, geralmente sendo executados no mesmo processo.

Esse tipo de arquitetura foi muito utilizada com Mainframes, que eram acessados por "terminais burros", computadores de baixo poder de processamento que apenas serviam como interface de acesso. Veja na figura a seguir:

11225354498 - Maria Vitória Gonçalves Vieira



Embora a arquitetura monolítica possa parecer simples e direta, ela também apresenta vários problemas:

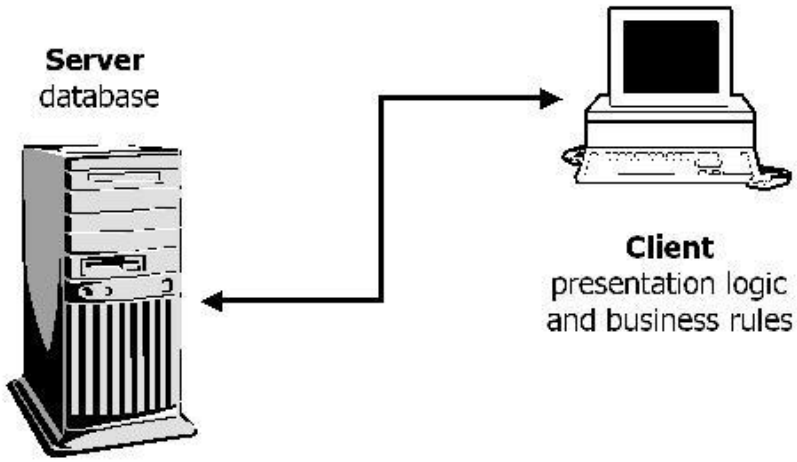
- **Dificuldade de Manutenção e Atualização:** Qualquer mudança, por menor que seja, exige que todo o sistema seja recompilado e redistribuído.
- **Escalabilidade Limitada:** Escalar um aplicativo monolítico geralmente requer a duplicação de todo o aplicativo, não apenas das partes que exigem mais recursos.
- **Falha Catastrófica:** Se um componente falhar, todo o sistema pode falhar, pois todos os componentes são interdependentes.
- **Complexidade Crescente:** À medida que o sistema cresce, sua complexidade também aumenta, tornando cada vez mais difícil entender e modificar o código.

Ao longo do tempo, com a expansão do tamanho e da complexidade dos sistemas de software, a arquitetura monolítica começou a mostrar suas limitações. Isso levou ao desenvolvimento de estilos de arquitetura mais modularizados e distribuídos, como a arquitetura em duas camadas.

Arquitetura em Duas Camadas

Com a popularização dos PCs na década de 1980 e início dos anos 1990, surgiu a necessidade de uma arquitetura que permitisse a separação entre o processamento de dados, geralmente realizada pelo servidor, e a interface do usuário, gerenciada pelo PC do cliente. Isso levou ao surgimento da arquitetura em duas camadas, também conhecida como arquitetura cliente-servidor.

11225354498 - Maria Vitória Gonçalves Vieira



Na arquitetura em duas camadas, a aplicação é dividida em duas partes principais:

1. **Camada do Cliente:** É a primeira camada desta arquitetura e é responsável pela interface do usuário. Ela é instalada na máquina do usuário (PC) e se encarrega da apresentação dos dados e da interação com o usuário.
2. **Camada do Servidor:** A segunda camada é o servidor, que geralmente é um Sistema de Gerenciamento de Banco de Dados (SGBD). Ele se encarrega de armazenar, recuperar e manipular os dados solicitados pelo cliente.

Nesse modelo, o cliente solicita serviços ao servidor, que executa a lógica de negócios e interage com o banco de dados para atender à solicitação. Os resultados são então retornados ao cliente para apresentação ao usuário.

Benefícios da Arquitetura em Duas Camadas:

- **Simplicidade:** O design é bastante direto, com uma divisão clara de responsabilidades entre o cliente e o servidor.
- **Distribuição de Carga de Trabalho:** A carga de trabalho é distribuída entre várias máquinas, pois o cliente se encarrega da apresentação dos dados e o servidor do processamento.

Limitações da Arquitetura em Duas Camadas:

- **Escalabilidade:** Como o servidor é responsável por processar todas as solicitações, pode se tornar um gargalo se o número de clientes for muito alto.
- **Manutenção:** Qualquer alteração na lógica de negócios requer a modificação do servidor e pode exigir a atualização do software do cliente.

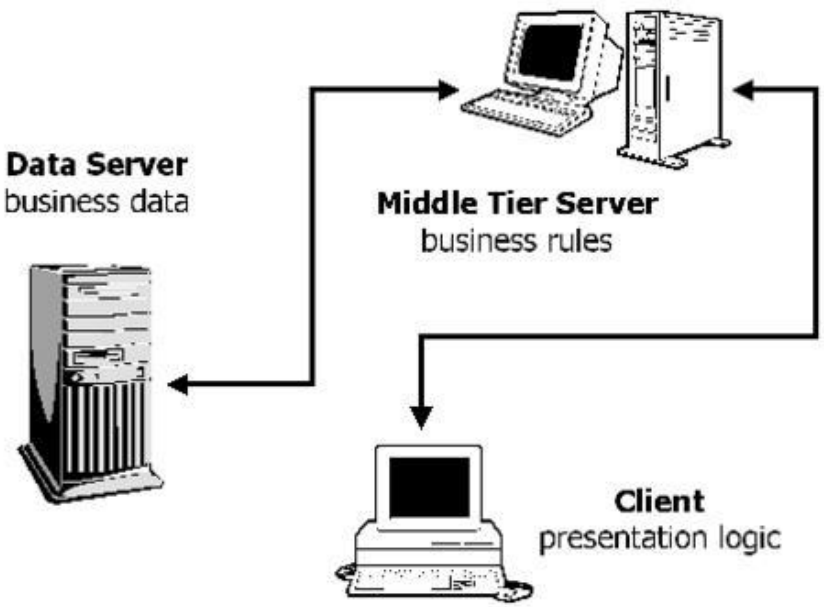
11225354498 - Maria Vitória Gonçalves Vieira

- **Distribuição de Carga de Trabalho Desigual:** O servidor normalmente tem mais carga de trabalho, pois é responsável pelo processamento da lógica de negócios e interação com o banco de dados.

Essas limitações, eventualmente, levaram ao desenvolvimento da arquitetura em três camadas, que adicionou uma camada intermediária de lógica de negócios entre a camada do cliente e do servidor, para melhor distribuição da carga de trabalho e maior flexibilidade.

Arquitetura em Três Camadas

A arquitetura em três camadas, também conhecida como arquitetura three-tier ou multicamadas, é uma extensão da arquitetura em duas camadas. Ela separa a lógica de negócios do cliente e do servidor, colocando-a em uma camada intermediária.



As três camadas na arquitetura em três camadas são:

1. **Camada de Apresentação:** É a camada superior e é a interface com a qual os usuários interagem. Esta camada pode ser um navegador web, uma interface gráfica de usuário (GUI) ou até mesmo uma interface de linha de comando. Ela é responsável pela exibição de informações ao usuário e pela coleta de informações do usuário.
2. **Camada de Negócio:** Também conhecida como camada de lógica de negócios ou camada de processamento, esta camada é onde as regras de negócios são executadas, as decisões são tomadas, e as funcionalidades do software são implementadas. A lógica de negócios é o código que fornece valor ao usuário. É aqui que os algoritmos são aplicados e os dados são manipulados para realizar algo útil.

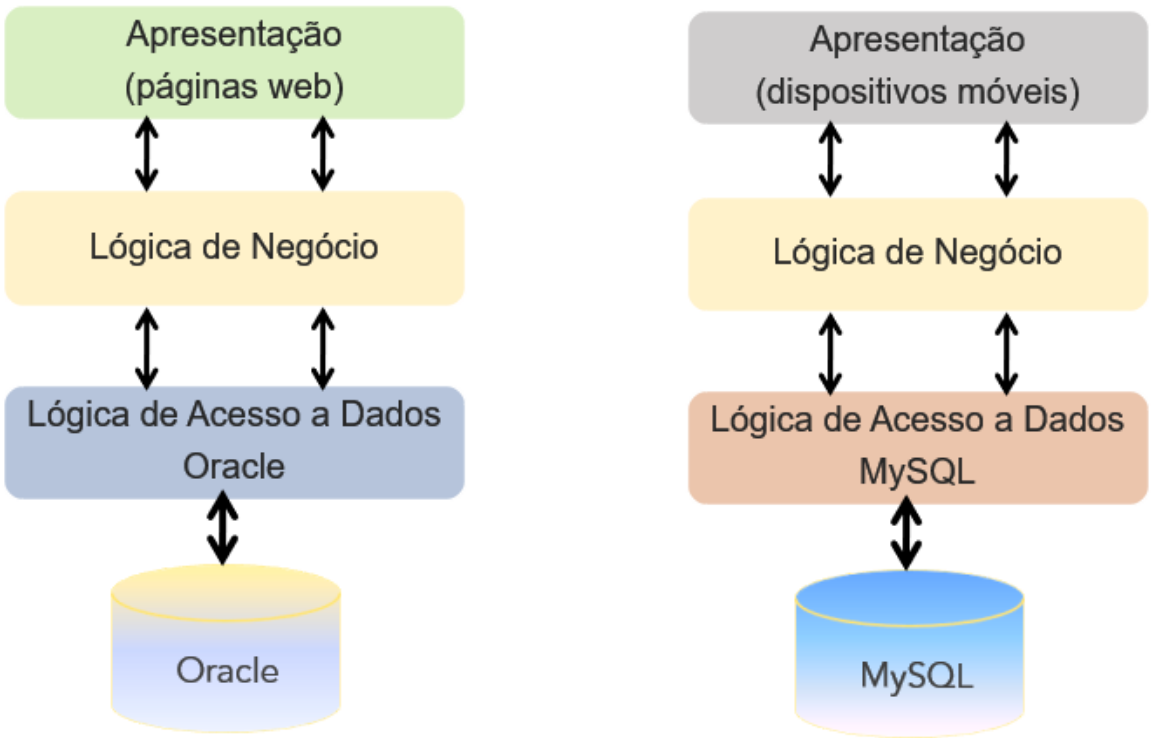
11225354498 - Maria Vitória Gonçalves Vieira

3. **Camada de Acesso a Dados:** A camada inferior é a camada de acesso a dados, onde os dados são salvos e recuperados do armazenamento. Esta camada interage com o banco de dados ou qualquer outro tipo de armazenamento de dados - seja um arquivo XML, um arquivo de texto, etc.

Benefícios da Arquitetura em Três Camadas:

1. **Separação de Responsabilidades:** Cada camada tem um propósito específico, tornando o sistema mais organizado e flexível.
2. **Manutenibilidade:** As mudanças em uma camada não afetam as outras camadas. Por exemplo, mudar a lógica de negócios não afeta a camada de apresentação ou a camada de acesso a dados.
3. **Reusabilidade:** As camadas podem ser reutilizadas em diferentes projetos. Por exemplo, uma camada de acesso a dados pode ser reutilizada por diferentes aplicativos.
4. **Escalabilidade:** Cada camada pode ser escalada horizontalmente de forma independente.

Veja como as camadas podem ser substituídas com diferentes tecnologias:



Agora veja um exemplo de tecnologias que podem ser aplicadas a cada camada:



Por fim, é importante notar que a arquitetura em três camadas também pode ter suas desvantagens, como a maior complexidade devido à necessidade de gerenciar a comunicação entre camadas e o fato de que a lógica de negócios pode se tornar um gargalo se não for projetada para lidar com a escalabilidade.

Arquitetura Web

A arquitetura web moderna é construída em cima da arquitetura em três camadas que mencionamos antes. As camadas são as seguintes:

1. **Camada de Apresentação:** Esta é a interface do usuário da aplicação web e é normalmente entregue ao usuário através de um navegador web. A camada de apresentação é escrita em HTML, CSS, e JavaScript e é responsável por exibir informações ao usuário e coletar a entrada do usuário.
2. **Camada de Aplicação:** A camada de negócio é onde a lógica de negócios da aplicação é implementada. Essa lógica pode ser implementada em um servidor web utilizando várias linguagens de programação, como Java, Python, Ruby, PHP, JavaScript (Node.js), etc.
3. **Camada de Acesso a Dados:** A camada de acesso a dados é responsável por interagir com o banco de dados ou outros serviços de armazenamento para salvar e recuperar dados.

Nesta arquitetura, o **navegador funciona como um cliente universal**. Isto é, o navegador é capaz de processar e exibir conteúdo de qualquer servidor web do mundo, desde que esse conteúdo esteja em formatos padrão da web, como HTML, CSS e JavaScript.

No caso de arquitetura Web, pode haver mais do que três camadas, bastando adicionar quantos servidores forem necessários, tais como servidor de mensageria, servidor web, servidor de aplicação, etc.

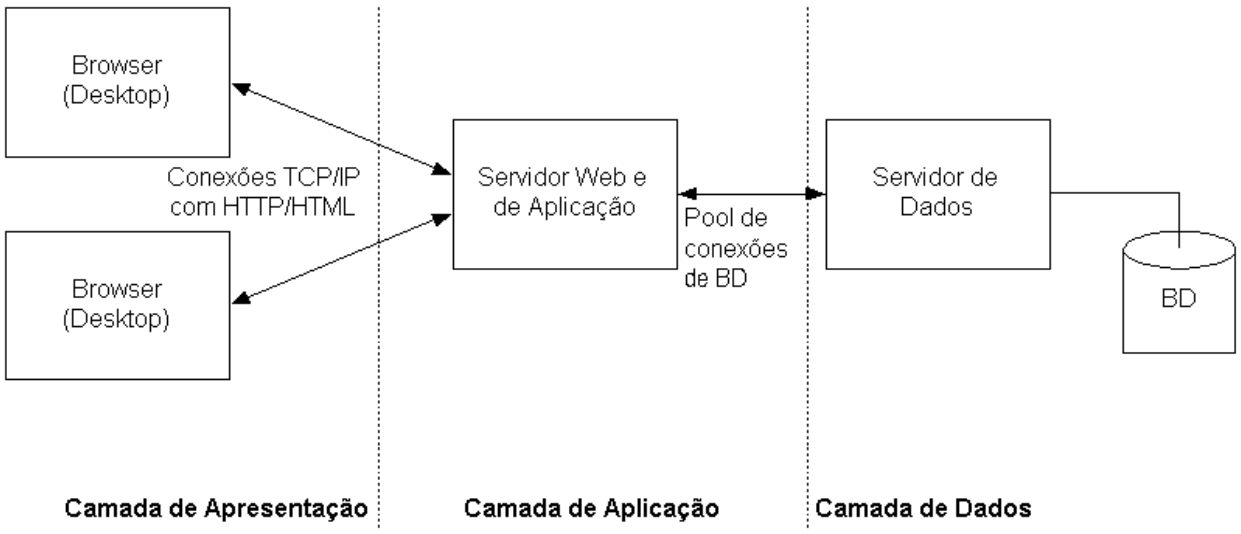
Aliás, muita gente não sabe a diferença entre um servidor Web e um servidor de Aplicação. Veja:

Um **servidor web** é responsável por receber solicitações de clientes (navegadores) e fornecer as páginas web correspondentes. Ele gerencia protocolos de comunicação, como HTTP, e lida com solicitações para recursos estáticos, como arquivos HTML, CSS, imagens e scripts. O servidor web retorna esses recursos para o cliente, que os exibe em um navegador.

Um **servidor de aplicação** é responsável por hospedar e executar aplicativos de software complexos. Ele fornece um ambiente em tempo de execução para a execução de código (Java, PHP, Python, etc.) de aplicativo e oferece suporte a recursos avançados, como lógica de negócio, persistência de dados, segurança e gerenciamento de transações.

Em outras palavras, um servidor Web responde a requisições HTTP, geralmente servindo páginas HTML e recursos associados (CSS, Javascript, etc.). Já um servidor de Aplicação é quem realmente executa o código de negócio de um sistema.

A imagem abaixo ilustra uma possível configuração de arquitetura web:



11225354498 - Maria Vitória Gonçalves Vieira

Arquitetura MVC

O Modelo-Visão-Controlador (Model-View-Controller) é um padrão de arquitetura de software que separa uma aplicação em três componentes principais: o Modelo, a Visão e o Controlador. Este padrão é frequentemente usado em aplicações web, onde a divisão em componentes facilita a manutenção e o teste do código. Vamos examinar cada um desses componentes.

Visão (View):

A visão é a **representação visual dos dados do modelo**. Ela é responsável por exibir os dados ao usuário em um formato legível. A visão pode ser qualquer tipo de saída de representação de dados, como um gráfico, um diagrama ou uma tabela. Muitas vezes, uma aplicação pode ter várias visões para o mesmo modelo, como uma visão detalhada e uma visão resumida.

Além disso, essa é a camada **responsável pela interação com o usuário**, sendo responsável apenas pela exibição de dados, ou seja, ela não sabe nada sobre o que a aplicação está fazendo atualmente, pois apenas recebe instruções do controle, notifica o controle e recebe informações do modelo.

Controlador (Controller):

Essa é a camada responsável por receber todas as requisições do usuário. Seus métodos – chamados actions – são responsáveis por uma página, controlando qual modelo usar e qual visão será mostrada ao usuário. Ele é **capaz de enviar comandos para o modelo atualizar o seu estado**. **Ele também pode enviar comandos para a respectiva visão para alterar a apresentação da visão do modelo**.

A Camada de Controle **atende às requisições do usuário e seleciona o modelo e a visão que o usuário usará para interagir com o modelo**. O usuário interage com controladores – por meio de visões – que interpretam eventos e entradas enviadas (Input), mapeando ações do usuário em comandos que são enviados para o modelo e/ou para a visão para efetuar as alterações apropriadas (Output).

Um controlador define o comportamento da aplicação, interpretando as ações do usuário e mapeando-as em chamadas do modelo. Em um cliente de aplicações web, essas ações do

11225354498 - Maria Vitória Gonçalves Vieira

usuário poderiam ser cliques em botões ou seleções de menus. As ações realizadas pelo modelo poderiam ser ativar processos de negócio ou alterar o estado do modelo.

O nome dessa camada se dá porque ela controla o fluxo da aplicação, interpretando os dados de entrada e coordenando/orquestrando as manipulações do modelo e as interações com o usuário. Trata-se de uma camada intermediária entre a Visão e o Modelo. Em geral, há um controlador para cada visão, apesar de poder existir várias controladoras para uma mesma visão.

Modelo (Model):

O modelo representa os dados e as regras de negócio da aplicação. É aqui que as informações são armazenadas e recuperadas. O modelo não tem conhecimento da visão e do controlador; ele simplesmente mantém os dados e fornece métodos para acessá-los e manipulá-los.

A Camada de Modelo encapsula as principais funcionalidades e dados do sistema. Ela notifica suas visões e respectivos controladores quando surge alguma mudança em seu estado, isto é, ela é responsável pela manutenção do estado da aplicação. Estas notificações permitem que as visões produzam saídas atualizadas e que os controladores alterem o conjunto de comandos disponíveis.

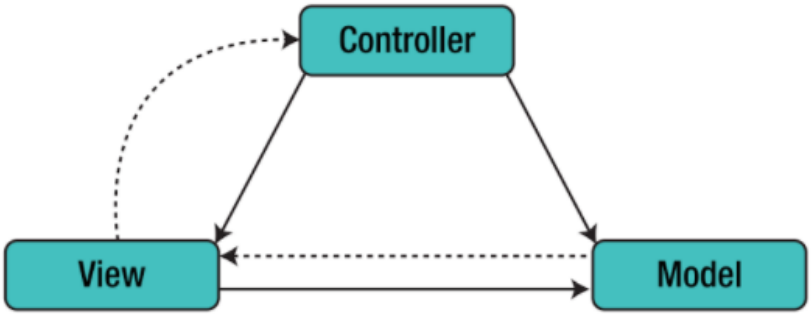
Interações no MVC

À primeira vista, a Arquitetura MVC parece não ter diferença alguma em relação à Arquitetura em Três-Camadas, com o Modelo substituindo a Camada de Dados, a Visão substituindo a Camada de Apresentação e o Controlador substituindo a Camada de Lógica de Negócio. **No entanto, essas duas arquiteturas são diferentes em relação a interação entre suas camadas**.

Na Arquitetura em Três-Camadas, a comunicação entre camadas é **rigidamente linear**, isto é, a Camada de Apresentação e a Camada de Dados só se conversam bidirecionalmente com a Camada de Lógica, mas nunca entre si. **Já no MVC, a comunicação é triangular** – existem diversas implementações diferentes dessa arquitetura, uma comunicação típica é apresentada na imagem a seguir.

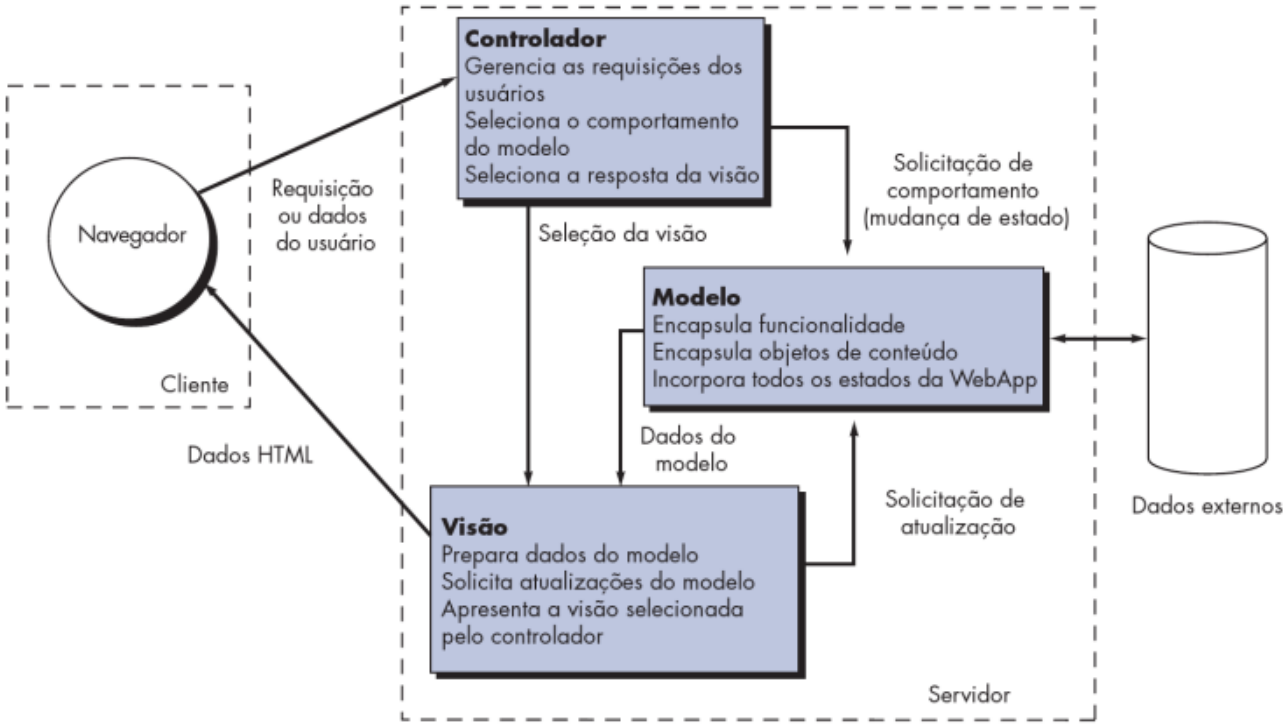
Observe que a Visão pode tanto gerar eventos a serem tratados pelo Controle quanto obter os dados a serem exibidos diretamente do modelo. O Controle trata os eventos da Visão, mas também pode manipular diretamente o Modelo. Finalmente, o Modelo pode reagir diretamente tanto à Visão quanto ao Controle, mas também pode gerar eventos a serem tratados pela visão.

11225354498 - Maria Vitória Gonçalves Vieira



Essa última sentença é extremamente polêmica – vocês encontrarão muitos lugares dizendo que não é possível que a visão solicite diretamente o estado do modelo, mas é possível, sim! Vamos supor que você esteja comprando cursos no site do Estratégia. Colocou um no carrinho, colocou dois e colocou o terceiro. Você, então, clica no botão de listar os itens que estão no carrinho.

Nesse momento, a visão não precisa necessariamente fazer uma requisição para o controle, para que o controle peça a lista de itens para o modelo. Ora, ela **pode pedir diretamente para o modelo**! Fiquem ligados também que existem diversas visões para cada modelo.



Ademais, não podemos confundir MVC com MVP (Model-View-Presenter). O MVP é uma evolução do MVC que se comunica bidirecionalmente com as outras camadas, evitando que o Model tenha que se comunicar diretamente com a View sem passar pelo Controller e este último é fundamental para a interação com o usuário. O MVP desacopla as funções e torna a arquitetura ainda mais modular.

11225354498 - Maria Vitória Gonçalves Vieira

A camada Presenter é ciente de tudo o que ocorre nas outras duas camadas e deixa-as cientes do que ela está fazendo; a interação com usuário é feita primariamente pela View, e esta pode delegar ao Presenter determinadas tarefas; há uma relação um-para-um entre estas camadas. Nesta relação, há uma referência do View para o Presenter mas não o oposto.

Finalmente, não devemos confundir também com o MVVM (Model-View-ViewModel). O MVVM é uma pequena evolução do MVP em um lado e um retrocesso em outro. Nele o ViewModel não está ciente do que ocorre no View, mas este está ciente do que ocorre no ViewModel (como no Padrão de Projeto Observer). No caso do Model, ambos estão cientes do que ocorre em cada um.

O nome se dá porque ele adiciona propriedades e operações ao Model para atender as necessidades do View, portanto ele cria um novo modelo para a visualização. É possível associar várias Views para um ModelView. É comum que as Views sejam definidas de forma declarativa (HTML/CSS, XAML, etc.). O Data Binding é feito entre a View e o ViewModel.

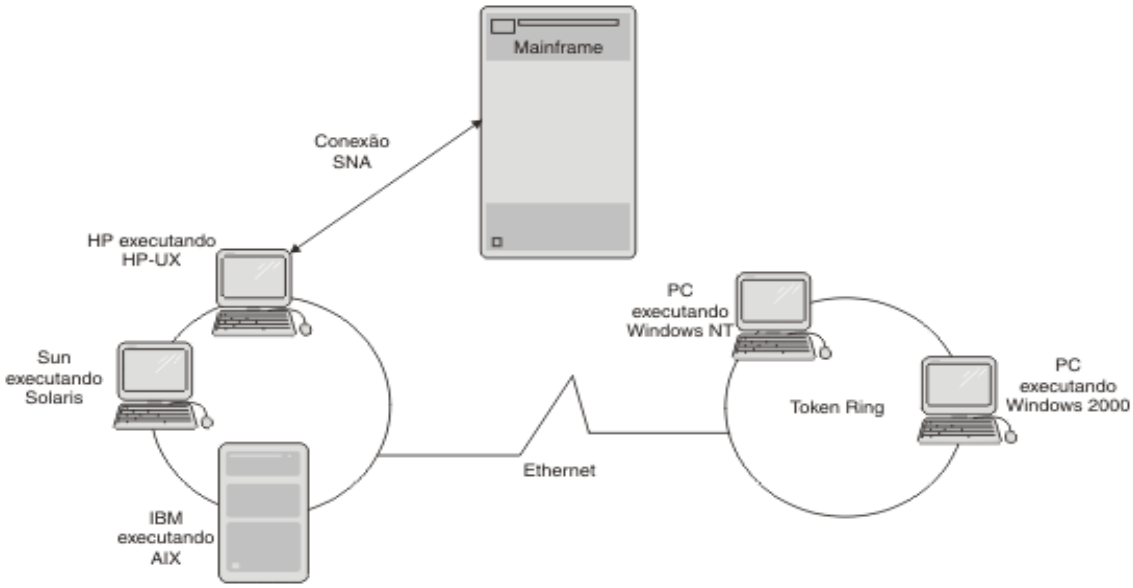
Com esse padrão é possível reduzir a quantidade de código para manter. Algumas automações são possíveis por ter todas as informações necessárias no ViewModel. ViewModel – é só um modelo mais adequado para uma visão específica (ou mais que uma).

Arquitetura Distribuída

A arquitetura distribuída é uma arquitetura de software na qual **componentes de um sistema de software são compartilhados entre várias máquinas, que se comunicam e coordenam suas ações por passagem de mensagens**. Cada componente é responsável por uma tarefa específica e opera de forma independente das outras.

Os sistemas distribuídos podem ser construídos de várias formas. Alguns exemplos incluem a arquitetura cliente-servidor, a arquitetura em três camadas, arquitetura baseada em serviços (como a arquitetura de microsserviços), e arquitetura peer-to-peer. Veja na figura abaixo como cada componente da arquitetura pode ser de uma tecnologia diferente. A heterogeneidade de tecnologias é uma característica da arquitetura distribuída.

11225354498 - Maria Vitória Gonçalves Vieira



A ideia é que o usuário não perceba que o processamento se dá de maneira distribuída. Essa percepção é transparente para o usuário**. A meta da computação distribuída é fazer o trabalho de rede como um computador único** (ainda que estejam geograficamente espalhados)

Vantagens da computação distribuída:

- **Desempenho:** Ao distribuir tarefas por vários nós, um sistema pode aproveitar a capacidade de processamento de várias máquinas e, assim, processar dados mais rapidamente do que um único nó poderia.
- **Escalabilidade:** Quando a carga de trabalho aumenta, novos nós podem ser adicionados ao sistema para ajudar a lidar com a carga adicional.
- **Disponibilidade:** Se um nó falhar, outros nós podem assumir a sua carga de trabalho, garantindo que o sistema continue a funcionar.
- **Tolerância a falhas:** Sistemas distribuídos são mais capazes de lidar com falhas, pois cada nó opera de forma independente. Se um nó falhar, isso não causará a falha do sistema como um todo.

No entanto, os sistemas distribuídos também apresentam dificuldades. A coordenação entre diferentes nós pode ser complexa e os problemas de rede podem afetar o desempenho do sistema. Além disso, garantir a consistência dos dados em todo o sistema pode ser difícil, especialmente quando vários nós estão modificando dados ao mesmo tempo.

Por fim, é importante notar que a comunicação na arquitetura distribuída é facilitada por **Middlewares**. Um Middleware é **qualquer camada de software que permite a interoperabilidade entre elementos de aplicações** e um modo de enviar ou obter dados de um lugar para o outro. É capaz de mascarar diferenças existentes entre sistemas

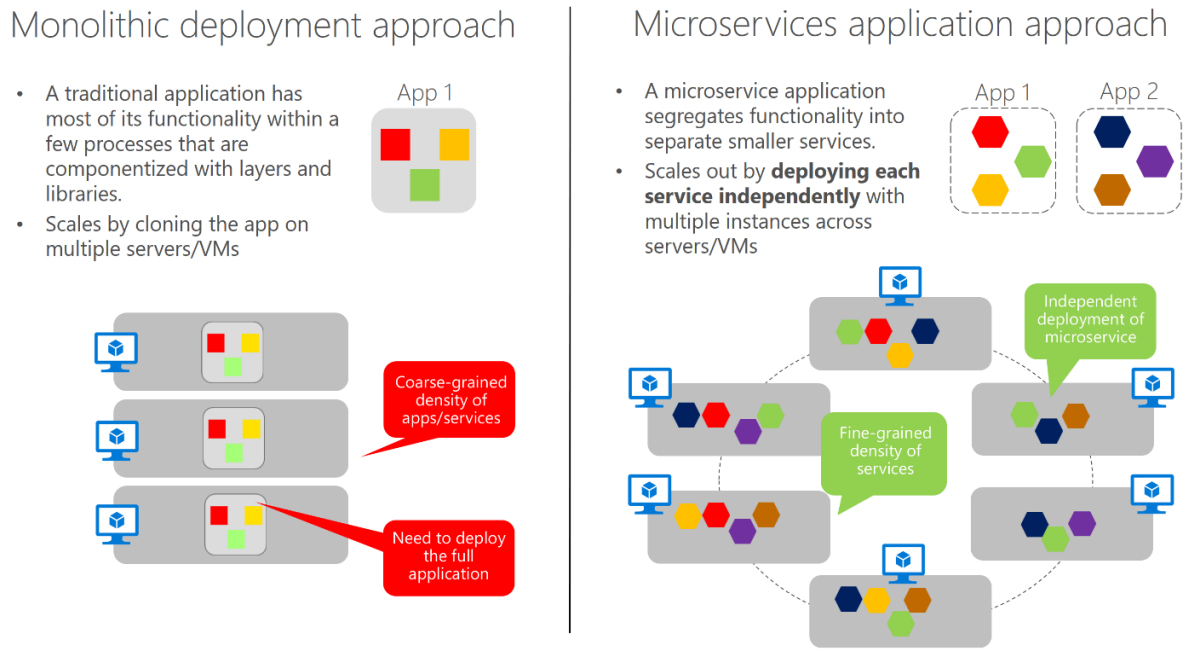
11225354498 - Maria Vitória Gonçalves Vieira

operacionais, plataformas de hardware e rede, dentro outras. Exemplos de Middlewares famosos incluem CORBA, RMI (Remote Method Invocation, em Java), RPC (Remote Procedure Call, em C/C++) e ODBC para bancos de dados.

Arquitetura de Microsserviços

A arquitetura de microsserviços é uma **abordagem para desenvolver um único aplicativo como uma suíte de pequenos serviços, cada um executando um processo único e comunicando-se por meio de mecanismos leves**, muitas vezes uma API HTTP/REST. Cada microsserviço é construído em torno de uma capacidade de negócios específica e pode ser desenvolvido independentemente por uma equipe pequena e focada.

A figura abaixo mostra as diferenças entre uma arquitetura monolítica (tradicional) e uma arquitetura em microsserviços:



Veja algumas características fundamentais dessa arquitetura:

- **Descentralização:** A arquitetura de microsserviços favorece a descentralização, tanto no que diz respeito ao armazenamento de dados, onde cada serviço mantém seu próprio banco de dados, quanto à descentralização do processo de tomada de decisões, onde as equipes têm autonomia para tomar decisões técnicas.

- **Isolamento de falhas:** Em uma arquitetura de microsserviços, os serviços são isolados uns dos outros. Isso significa que se um serviço falhar, ele não levará a uma falha de todo o sistema.
- **Independência de desenvolvimento e implantação:** Cada microsserviço é independente dos outros e pode ser desenvolvido, implantado e escalado de forma independente. Isso proporciona maior flexibilidade e velocidade no ciclo de desenvolvimento de software.
- **Tecnologia heterogênea:** Como cada microsserviço é independente, diferentes serviços podem ser escritos em diferentes linguagens de programação, usar diferentes estruturas de armazenamento de dados e ser executados em diferentes tipos de hardware.
- **Comunicação baseada em APIs:** Os microsserviços se comunicam uns com os outros através de APIs bem definidas, muitas vezes usando protocolos baseados em HTTP/REST ou mensagens de fila.

Porém, nessa arquitetura, existe uma complexidade para gerir múltiplos serviços, a necessidade de coordenação e governança de serviços, o risco de latência de rede e problemas de consistência de dados entre serviços. Esses obstáculos requerem um alto nível de maturidade operacional e experiência em desenvolvimento de software para gerir efetivamente.

A tabela a seguir resume algumas diferenças entre arquiteturas tradicionais e arquitetura de microsserviços:

Tradicional	Microsserviços
Aplicação é uma única unidade lógica	**Aplicações são conjuntos de serviços**
Ciclos de mudanças são altamente acoplados, i.e., uma mudança feita para uma pequena parte da aplicação requer que um novo build e um novo deploy da aplicação inteira.	**Implantações (deploys) independentes**
Normalmente escrito em uma única linguagem de programação	**Pode envolver tecnologias e linguagens heterogêneas**

11225354498 - Maria Vitória Gonçalves Vieira

Exemplo de Arquitetura de Microsserviço

Vamos tomar como exemplo uma aplicação de e-commerce. Em uma arquitetura monolítica, todas as funcionalidades, como gerenciamento de usuários, catálogo de produtos, carrinho de compras, pagamento e envio estariam todas em um único aplicativo. Na arquitetura de microsserviços, cada uma dessas funcionalidades seria um serviço separado. Veja como poderia ser:

- **Serviço de Usuários:** Gerencia a criação e autenticação de usuários. Quando um usuário se registra ou faz login, esse serviço é acionado.
- **Serviço de Catálogo de Produtos:** Este serviço gerencia a lista de produtos disponíveis. Ele é responsável por adicionar novos produtos, atualizar detalhes do produto e remover produtos.
- **Serviço de Carrinho de Compras:** Este serviço permite que os usuários adicionem produtos ao seu carrinho de compras, modifiquem a quantidade de um item no carrinho ou removam um item do carrinho.
- **Serviço de Pagamento:** Este serviço processa o pagamento. Ele se comunica com o gateway de pagamento para autorizar a transação.
- **Serviço de Envio:** Este serviço coordena o envio de produtos. Ele rastreia o status do pedido e atualiza o usuário com informações de rastreamento.

Cada um desses microsserviços pode ser desenvolvido, testado, implantado e escalado de forma independente. Por exemplo, se a aplicação precisa processar um grande número de pagamentos durante uma venda, o serviço de pagamento pode ser escalado para lidar com a carga extra, sem a necessidade de escalar outros serviços. Além disso, se um novo método de pagamento precisa ser implementado, isso pode ser feito no serviço de pagamento sem afetar os outros serviços.

A comunicação entre os microsserviços seria feita através de APIs, usando um protocolo como HTTP/REST ou mensagens assíncronas através de um sistema de mensageria.

Arquitetura de Mainframe

Um mainframe é um tipo de computador de **grande porte destinado ao processamento de grandes volumes de dados**. Esses sistemas são usados principalmente por grandes organizações para aplicações críticas, transações em massa, gerenciamento e processamento de dados, como censos, transações financeiras da indústria e de consumo, processamento de transações de cartão de crédito e interações de telecomunicações.

11225354498 - Maria Vitória Gonçalves Vieira

A arquitetura mainframe tem várias características que a tornam ideal para esses tipos de aplicações:

- **Alto Desempenho:** Os mainframes são projetados para lidar com altas cargas de trabalho e podem processar milhares de transações por segundo.
- **Escalabilidade:** São altamente escaláveis, permitindo que as organizações aumentem a capacidade de processamento conforme necessário.
- **Disponibilidade:** são projetados para serem extremamente confiáveis e disponíveis. Eles incluem características como redundância de hardware e recursos avançados de recuperação de falhas para garantir que o sistema permaneça disponível mesmo em caso de falhas de hardware ou software.
- **Segurança:** Mainframes também são conhecidos por suas fortes capacidades de segurança, tornando-os ideais para aplicações que lidam com dados sensíveis.
- **Compatibilidade com versões anteriores:** Uma das grandes vantagens dos mainframes é a sua capacidade de manter a compatibilidade com versões anteriores. Isso significa que as aplicações desenvolvidas para mainframes mais antigos ainda podem funcionar nos mainframes modernos.

Desvantagens de mainframes:

- **Custo Elevado:** Os mainframes são máquinas de alto desempenho e, portanto, podem ser muito caros. Isso inclui não apenas o custo inicial de aquisição do hardware, mas também os custos contínuos de manutenção, atualização e energia.
- **Flexibilidade Limitada:** Os mainframes não são tão flexíveis quanto as arquiteturas modernas baseadas em microsserviços ou em nuvem. As mudanças são mais difíceis de implementar e geralmente requerem mais planejamento e tempo.
- **Escassez de Habilidades:** Como a tecnologia de mainframe é mais antiga, pode ser mais difícil encontrar desenvolvedores e administradores de sistema com as habilidades necessárias. Isso pode ser especialmente desafiador à medida que a força de trabalho mais antiga se aposenta e a nova geração de profissionais de TI é mais familiarizada com tecnologias mais recentes.
- **Integração com Novas Tecnologias:** Integrar mainframes com novas tecnologias pode ser um desafio, pois eles muitas vezes usam sistemas e protocolos mais antigos que podem não ser compatíveis com as tecnologias mais recentes.
- **Modernização:** Modernizar um sistema de mainframe para usar tecnologias mais recentes pode ser um projeto complexo e caro. No entanto, muitas organizações estão adotando abordagens como a encapsulação, que permite a eles expor a lógica de negócios de mainframe como APIs para serem usadas por aplicações mais modernas.

Talvez o caso de uso mais comum para Mainframes, especialmente no Brasil, é no setor de Bancos e Instituições Financeiras. Estas instituições lidam com milhões de transações por dia, todas as quais precisam ser processadas e registradas de forma eficiente e segura. Os

11225354498 - Maria Vitória Gonçalves Vieira

mainframes são usados para processar transações de cartões de crédito, transferências bancárias, transações de bolsa de valores e muito mais.

Um outro caso comum é de aplicações de governo, a maior delas sendo o SIAFI - Sistema de Administração Financeira do Governo Federal - gerenciado pela Secretaria do Tesouro Nacional (*onde sou Auditor Federal há mais de uma década e liderei pessoalmente o projeto de modernização do SIAFI* =))

Por fim, não se engane em achar mainframes uma tecnologia antiquada em comparação com a arquitetura orientada a serviços e a computação em nuvem, pois eles ainda são uma parte muito importante da infraestrutura de TI de muitas indústrias, devido à sua confiabilidade, capacidade de processamento e segurança.

Arquitetura Peer-to-Peer (P2P)

Uma arquitetura peer-to-peer (P2P) é um modelo de sistema distribuído em que **os participantes têm papéis e responsabilidades semelhantes**, permitindo a comunicação e compartilhamento de recursos de forma descentralizada. Ao contrário de arquiteturas tradicionais cliente-servidor, em que um servidor centralizado desempenha um papel dominante no fornecimento de serviços, a arquitetura P2P permite que os participantes atuem tanto como clientes quanto como servidores.

Veja algumas características dessa arquitetura:

- **Descentralização**: Uma arquitetura P2P é descentralizada, o que significa que não há uma autoridade central controlando o sistema. Cada nó participante tem autonomia e pode se conectar diretamente com outros nós, sem depender de um servidor central.
- **Igualdade de papéis**: Todos os nós em uma rede P2P têm papéis e responsabilidades semelhantes. Cada nó pode atuar como um cliente, solicitando recursos ou serviços, e também como um servidor, fornecendo recursos ou serviços para outros nós.
- **Escalabilidade**: A arquitetura P2P é altamente escalável. O número de nós na rede pode variar e a adição de novos nós aumenta a capacidade de processamento e armazenamento disponível na rede. Isso permite que a rede se adapte ao crescimento e suporte uma grande quantidade de participantes.
- **Redundância e tolerância a falhas**: Em uma arquitetura P2P, os recursos e serviços são distribuídos entre os nós participantes. Isso cria redundância, pois os recursos estão disponíveis em vários nós. Se um nó falhar, outros nós ainda podem fornecer o recurso necessário, aumentando a tolerância a falhas.

11225354498 - Maria Vitória Gonçalves Vieira

- **Eficiência**: Uma arquitetura P2P pode ser eficiente em termos de utilização de recursos. Por exemplo, em sistemas de compartilhamento de arquivos P2P, o download de um arquivo pode ocorrer simultaneamente a partir de várias fontes (nós), aproveitando a largura de banda disponível em diferentes partes da rede.
- **Autoorganização**: Os nós em uma rede P2P podem se autoorganizar, descobrindo e se conectando a outros nós sem a necessidade de uma configuração centralizada. Isso permite que a rede cresça e se adapte dinamicamente, conforme os nós entram ou saem da rede.
- **Privacidade e segurança**: Uma arquitetura P2P pode fornecer maior privacidade e segurança, uma vez que não depende de um servidor central que possa ser alvo de ataques ou monitoramento. Além disso, a distribuição dos dados entre vários nós dificulta o acesso não autorizado aos recursos.

Existem várias arquiteturas peer-to-peer (P2P) famosas que têm sido amplamente utilizadas ao longo dos anos. Aqui estão alguns exemplos notáveis:

1. **BitTorrent**: O BitTorrent é um dos protocolos P2P mais conhecidos e amplamente utilizados para compartilhamento de arquivos. Ele permite que os usuários façam o download e o upload de partes de um arquivo simultaneamente de vários pares (peers) na rede, aproveitando a largura de banda coletiva dos participantes.
2. **Napster**: O Napster foi um dos primeiros sistemas P2P populares, desenvolvido no final dos anos 1990. Ele permitia que os usuários compartilhassem arquivos de música diretamente entre si. O Napster introduziu o conceito de índice centralizado, em que um servidor central mantinha um registro dos arquivos disponíveis nos nós da rede.
3. **eDonkey2000**: O eDonkey2000 era um sistema P2P popular para compartilhamento de arquivos, especialmente arquivos grandes. Ele usava uma combinação de redes descentralizadas e servidores centrais para facilitar a descoberta e a transferência de arquivos.
4. **Blockchain**: Embora não seja estritamente uma arquitetura P2P, a tecnologia blockchain utiliza princípios P2P para criar redes descentralizadas. No blockchain, os nós da rede trabalham juntos para validar e registrar transações, criando um sistema confiável e seguro sem depender de uma autoridade central.

APOSTA ESTRATÉGICA

A ideia desta seção é apresentar os pontos do conteúdo que mais possuem chances de serem cobrados em prova, considerando o histórico de questões da banca em provas de nível

11225354498 - Maria Vitória Gonçalves Vieira

semelhante à nossa, bem como as inovações no conteúdo, na legislação e nos entendimentos doutrinários e jurisprudenciais[1].

O Modelo-Visão-Controlador (MVC) é um padrão de arquitetura de software amplamente adotado que organiza uma aplicação em três componentes interconectados: o Modelo, a Visão e o Controlador. Essa separação ajuda a gerenciar a complexidade das aplicações, facilitando a manutenção, a escalabilidade e o desenvolvimento. O Modelo é responsável por gerenciar os dados e as regras de negócio da aplicação. Ele comunica diretamente com o banco de dados para buscar e armazenar informações, e também define os dados que são relevantes para a aplicação. Ao centralizar as regras de negócio no Modelo, o MVC permite que as mudanças nos dados ou nas regras de negócio sejam feitas com mínimo impacto sobre a interface do usuário.

A Visão, por sua vez, é o componente que lida com a exibição dos dados, sendo a interface com a qual os usuários interagem. Normalmente, ela é composta de elementos gráficos que apresentam os dados aos usuários e capturam suas interações, que são então enviadas ao Controlador. O Controlador atua como um intermediário entre o Modelo e a Visão. Ele recebe as ações do usuário, geralmente através da interface, processa essas ações, possivelmente atualizando o Modelo, e retorna uma representação apropriada de dados, que é então exibida pela Visão. Este ciclo de interações facilita a separação de tarefas e a organização do código, permitindo que desenvolvedores e designers trabalhem de forma mais independente, focando em suas áreas de expertise sem interferir no trabalho um do outro.

QUESTÕES ESTRATÉGICAS

Nesta seção, apresentamos e comentamos uma amostra de questões objetivas selecionadas estrategicamente: são questões com nível de dificuldade semelhante ao que você deve esperar para a sua prova e que, em conjunto, abordam os principais pontos do assunto.

A ideia, aqui, não é que você fixe o conteúdo por meio de uma bateria extensa de questões, mas que você faça uma boa revisão global do assunto a partir de, relativamente, poucas questões

1. **(UFG / SANEAGO – 2017)** O emprego de boas práticas de projeto (design) de software visa resultar em um código:

[1] Vale deixar claro que nem sempre será possível realizar uma aposta estratégica para um determinado assunto, considerando que às vezes não é viável identificar os pontos mais prováveis de serem cobrados a partir de critérios objetivos ou minimamente razoáveis.

a) altamente acoplado e altamente coeso.

b) altamente acoplado e fracamente coeso.

c) fracamente acoplado e altamente coeso.

d) fracamente acoplado e fracamente coeso.

Comentários:

A regra de ouro de uma arquitetura de software: alta/forte coesão e baixo/fraco acoplamento!

Gabarito: Letra C

2. **(CESPE / TRE-BA – 2017)** Com referência às arquiteturas multicamadas de aplicações para o ambiente web, assinale a opção correta.

a) Se, na camada de dados, for realizada uma alteração no banco de dados, o restante das camadas também será afetado.

b) O modelo de três camadas recebe essa denominação caso um sistema cliente-servidor seja desenvolvido mantendo-se a camada de negócio do lado do cliente e as camadas de apresentação e dados no lado do servidor.

c) Cada camada é normalmente mantida em um servidor específico para tornar-se o mais escalonável e independente possível em relação a outras camadas, estando entre as suas principais características o eficiente armazenamento e a reutilização de recursos.

d) O objetivo das arquiteturas multicamadas consiste na junção de responsabilidades entre os componentes das aplicações web, de modo a atender aos requisitos funcionais e não funcionais esperados pela aplicação.

e) Na arquitetura de duas camadas — apresentação e armazenamento —, o computador que contiver a base de dados terá de ficar junto com os computadores que executarem as aplicações.

Comentários:

11225354498 - Maria Vitória Gonçalves Vieira

(a) Errado, uma alteração no banco de dados alteraria apenas as classes da camada de dados, mas o restante da arquitetura não seria afetado por essa alteração; (b) Errado, ela recebe essa denominação quando um sistema cliente-servidor é desenvolvido retirando-se a camada de negócio do lado do cliente; (c) Correto. Cada camada desta arquitetura é normalmente mantida em um servidor específico para tornar-se mais escalonável e independente das demais. Cada camada é autocontida o suficiente de forma que a aplicação pode ser dividida em vários computadores em uma rede distribuída; (d) Errado, o objetivo consiste na separação de responsabilidades entre os componentes das aplicações web, de modo que tenham alta coesão; (e) Errado. Nesta estrutura, a base de dados é colocada em uma máquina específica, separada das máquinas que executavam as aplicações.

Gabarito: Correto

3. **(IBFC / EBSERH – 2017)** O modelo de três camadas físicas (3-tier), especificado nas alternativas, divide um aplicativo de modo que a lógica de negócio resida no meio das três camadas, foi adaptado como uma arquitetura para as aplicações Web em todas as linguagens de programação maiores. Muitos frameworks de aplicação comerciais e não comerciais foram criados tendo como base a arquitetura:

   a) MVC (Model-View-Controller)

   b) MDB (Model-Data-Business)

   c) UDC (User-Data-Controller)

   d) MDC (Model-Data-Controller)

   e) UVB (User-View-Business).

   Comentários:

   A divisão do aplicativo que separa em três camadas é a Arquitetura MVC (Model, View e Control).

   Gabarito: Letra A

4. **(FCC / TCM-GO – 2015 – Adaptada)** Quanto à Arquitetura em 3 Camadas, é necessário um arranjo que possibilite a reutilização do código e facilite sua manutenção e seu aperfeiçoamento. Deve- se separar Apresentação, Regra de

Negócio e Acesso a Dados. Busca-se a decomposição de funcionalidades de forma a permitir aos desenvolvedores concentrarem-se em diferentes partes da aplicação durante a implementação.

Comentários:

Perfeito! Essa é uma situação bastante comum em uma arquitetura em três camadas.

Gabarito: Letra C

5. **(FCC / CNMP – 2015)** Há algumas variantes possíveis de arquitetura a serem utilizadas em um sistema de bancos de dados. Sobre essas variantes, é correto afirmar que:

a) na arquitetura de 3 camadas, não há uma camada específica para a aplicação.

b) a camada de apresentação da arquitetura de 2 camadas situa-se, usualmente, no servidor de banco de dados.

c) na arquitetura de 3 camadas, a camada de servidor de banco de dados é denominada cliente.

d) a arquitetura de 3 camadas é composta pelas camadas cliente, aplicação e servidor de banco de dados.

e) na arquitetura de 2 camadas não há necessidade de uso de um sistema gerenciador de bancos de dados.

Comentários:

(a) Errado, é a camada intermediária; (b) Errado, fica na camada de apresentação; (c) Errado, é chamada camada de dados; (d) Correto; (e) Errado, é claro que há necessidade.

Gabarito: Letra D

6. **(VUNESP - 2023 - TCM-SP - Auxiliar Técnico de Controle Externo - Técnico de Informática)**

A respeito do conceito de arquitetura de software em múltiplas camadas (multitier architecture), é correto afirmar que

11225354498 - Maria Vitória Gonçalves Vieira

A) as camadas de interface de usuário e lógica de negócio possuem separação conceitual e independência de implementação, mas suas implementações precisam ser executadas na mesma máquina. A camada de dados pode ser executada em outra máquina.

B) as camadas de lógica de negócio e de dados possuem separação conceitual e independência de implementação, mas suas implementações precisam ser executadas na mesma máquina. A camada de interface de usuário pode ser executada em outra máquina.

C) as implementações das diferentes camadas podem ser executadas em diferentes máquinas, podendo a aplicação estar dividida em mais de 3 camadas.

D) as implementações das diferentes camadas podem ser executadas em diferentes máquinas, mas a arquitetura possibilita, no máximo, 3 camadas.

E) todas as camadas possuem separação conceitual e independência de implementação entre si, e suas implementações precisam ser executadas na mesma máquina.

Comentários:

Na arquitetura de software em múltiplas camadas, também conhecida como arquitetura multitier, as diferentes camadas têm a capacidade de serem executadas em máquinas separadas. Essa arquitetura permite a divisão lógica e funcional do sistema em camadas distintas, como a camada de interface de usuário, a camada de lógica de negócio e a camada de dados, entre outras possíveis camadas, dependendo da complexidade do sistema.

A separação conceitual e a independência de implementação entre as camadas são características importantes dessa arquitetura, permitindo que cada camada seja desenvolvida, mantida e escalada de forma independente das outras camadas. Além disso, a distribuição das camadas em máquinas separadas pode trazer benefícios em termos de escalabilidade, desempenho e disponibilidade do sistema.

Gabarito: Letra C

7. (VUNESP - 2020 - Prefeitura de Ilhabela - SP - Analista - Tecnologia da Informação e Comunicação)

11225354498 - Maria Vitória Gonçalves Vieira

A respeito da arquitetura cliente-servidor, utilizada por muitas aplicações, é correto afirmar:

A) O programa servidor da aplicação precisa estar acessível via Internet, por meio de um endereço IP público e com a porta de acesso aberta nos firewalls aplicáveis

B) Um cliente pode ser acessado por um ou vários servidores, os quais fazem requisições de serviços ou funcionalidades aos primeiros.

C) A arquitetura P2P (peer-to-peer) é um sinônimo para a arquitetura cliente-servidor.

D) É uma arquitetura imune a ataques do tipo DDoS (Distributed Denial-of-Service) e, por isso, é bastante usada em aplicações que funcionam pela Internet.

E) Pode ser utilizada localmente, ou seja, o programa servidor e o programa cliente podem ser executados na mesma máquina.

Comentários:

Na arquitetura cliente-servidor, há uma divisão clara de papéis e responsabilidades entre o cliente e o servidor. O cliente é responsável por solicitar serviços ou funcionalidades ao servidor, que, por sua vez, é responsável por processar essas solicitações e fornecer as respostas correspondentes.

Uma das possibilidades dessa arquitetura é que tanto o programa cliente quanto o programa servidor sejam executados na mesma máquina, ou seja, localmente. Nesse caso, a comunicação entre o cliente e o servidor é feita internamente, sem a necessidade de acessar a Internet ou ter um endereço IP público.

Gabarito: Letra E

8. (VUNESP - 2020 - FITO - Técnico em Gestão - Informática)

Uma certa arquitetura de software de três camadas (three-tier) é composta pela camada de apresentação (presentation tier), camada de aplicação com lógica negócio (application/business tier) e camada de dados (data tier). Cada camada é implementada por um módulo independente do outro, podendo esses módulos ser executados em computadores diferentes, comunicando-se por TCP/IP. No contexto

11225354498 - Maria Vitória Gonçalves Vieira

dessa arquitetura e considerando os princípios desse padrão arquitetural, é correto afirmar que a camada de

A) aplicação funciona como um servidor para a camada de apresentação, e como um cliente para a camada de dados.

B) apresentação acessa diretamente a camada de dados, e a camada de aplicação faz o monitoramento dessa comunicação.

C) dados tem por finalidade apresentar dados da aplicação na interface do usuário, tal como exibir produtos em uma aplicação de comércio eletrônico.

D) aplicação utiliza diretamente um sistema gerenciador de banco de dados relacional (SGBDR) para manter os dados persistentes da aplicação.

E) apresentação é implementada por um navegador de Internet, razão pela qual essa arquitetura é restrita a aplicações web.

Comentários:

Na arquitetura de três camadas (three-tier), cada camada possui responsabilidades específicas e se comunica com as outras camadas de acordo com os princípios desse padrão arquitetural. Vamos analisar as alternativas.

A) Correto. A camada de aplicação funciona como um servidor para a camada de apresentação, fornecendo os serviços e funcionalidades necessárias para a interface do usuário. Nesse caso, a camada de apresentação age como cliente, solicitando os dados e serviços da camada de aplicação.

B) A camada de apresentação não acessa diretamente a camada de dados. Ela interage apenas com a camada de aplicação, que faz o monitoramento da comunicação entre a camada de apresentação e a camada de dados. A camada de aplicação atua como intermediária, realizando a lógica de negócio e gerenciando o acesso aos dados.

C) A camada de dados não tem como finalidade apresentar dados na interface do usuário. Sua responsabilidade é armazenar e gerenciar os dados persistentes da aplicação. A camada de apresentação é responsável pela exibição dos dados na interface do usuário.

11225354498 - Maria Vitória Gonçalves Vieira

D) Embora a camada de aplicação possa fazer uso de um sistema gerenciador de banco de dados relacional (SGBDR) para acessar e manter os dados persistentes da aplicação, isso não é uma afirmação correta sobre a arquitetura de três camadas em si. O acesso ao SGBDR é feito pela camada de aplicação, que atua como intermediária entre a camada de apresentação e a camada de dados.

E) A arquitetura de três camadas não é restrita a aplicações web. Embora seja comumente utilizada em aplicações web, onde a camada de apresentação pode ser implementada por um navegador de Internet, essa arquitetura também pode ser aplicada em outros tipos de aplicações, como aplicativos desktop ou móveis.

Gabarito: Letra A

9. (VUNESP - 2019 - Prefeitura de Itapevi - SP - Analista em Tecnologia da Informação e Comunicação)

No modelo de arquitetura Cliente-Servidor

A) um cliente não se comunica com outro cliente.

B) a função do servidor é receber as solicitações de serviços dos clientes receptores de serviços, e conectá-los aos clientes provedores de cada serviço requisitado.

C) caso o servidor não esteja em funcionamento, os clientes poderão solicitar serviços de outros clientes.

D) ocorre a comunicação direta entre os clientes, sem que haja uma separação fixa entre o cliente e o servidor.

E) um cliente envia uma solicitação de serviço pela rede, e outro cliente que pode atender executa o serviço, sob a supervisão do servidor.

Comentários:

No modelo de arquitetura cliente-servidor, a comunicação ocorre entre um cliente e um servidor. Um cliente não se comunica diretamente com outro cliente. A comunicação é estabelecida entre o cliente e o servidor, onde o cliente envia solicitações de serviços ao servidor e recebe as respostas correspondentes.

Gabarito: Letra A

11225354498 - Maria Vitória Gonçalves Vieira

10. (VUNESP - 2019 - UFABC - Técnico de Tecnologia da Informação)

Grande parte dos serviços da Internet utiliza a arquitetura Cliente/Servidor, pois apresenta, como vantagem,

A) a distribuição de solicitações nos Servidores, o que evita a sobrecarga.

B) a possibilidade de oferecer e disponibilizar serviços de forma espontânea

C) os dados serem armazenados no Servidor, o que melhora a segurança.

D) ser imune a falhas no Servidor, uma vez que há redundância de serviços.

E) utilizar um sistema operacional especializado, o que facilita a instalação.

Comentários:

A arquitetura Cliente/Servidor tem a vantagem de permitir que os dados sejam armazenados no servidor. Essa centralização dos dados pode melhorar a segurança, pois o servidor pode implementar mecanismos de controle de acesso, criptografia e backups, oferecendo uma camada adicional de proteção aos dados.

Gabarito: Letra C

11. (CESGRANRIO / CEFET-RJ – 2014) No contexto da Arquitetura de Sistemas, o MVC (model – view – controller) é um estilo arquitetural:

a) interativo

b) estrutural

c) distribuído

d) adaptável

e) monolítico

Comentários:

11225354498 - Maria Vitória Gonçalves Vieira

É um estilo arquitetural interativo, no sentido de que é um estilo para interface de usuário, fornecendo diversas visões diferentes para um mesmo modelo de dados.

Gabarito: Letra A

QUESTIONÁRIO DE REVISÃO E APERFEIÇOAMENTO

A ideia do questionário é elevar o nível da sua compreensão no assunto e, ao mesmo tempo, proporcionar uma outra forma de revisão de pontos importantes do conteúdo, a partir de perguntas que exigem respostas subjetivas.

São questões um pouco mais desafiadoras, porque a redação de seu enunciado não ajuda na sua resolução, como ocorre nas clássicas questões objetivas.

O objetivo é que você realize uma auto explicação mental de alguns pontos do conteúdo, para consolidar melhor o que aprendeu ;)

Além disso, as questões objetivas, em regra, abordam pontos isolados de um dado assunto. Assim, ao resolver várias questões objetivas, o candidato acaba memorizando pontos isolados do conteúdo, mas muitas vezes acaba não entendendo como esses pontos se conectam.

Assim, no questionário, buscaremos trazer também situações que ajudem você a conectar melhor os diversos pontos do conteúdo, na medida do possível.

É importante frisar que não estamos adentrando em um nível de profundidade maior que o exigido na sua prova, mas apenas permitindo que você compreenda melhor o assunto de modo a facilitar a resolução de questões objetivas típicas de concursos, ok?

Nosso compromisso é proporcionar a você uma revisão de alto nível!

Vamos ao nosso questionário:

11225354498 - Maria Vitória Gonçalves Vieira

Perguntas

1. O que é arquitetura de software?
2. O que é coesão em arquitetura de software?
3. O que é acoplamento em arquitetura de software?
4. O que é arquitetura em camadas?
5. O que é arquitetura monolítica?
6. O que é arquitetura em duas camadas?
7. O que é arquitetura em três camadas?
8. O que é a arquitetura MVC?
9. O que é arquitetura distribuída?
10. O que é middleware?
11. O que é arquitetura de microsserviços?
12. O que é arquitetura mainframe?
13. O que é arquitetura P2P?
14. Qual é a diferença entre uma arquitetura cliente-servidor e uma arquitetura P2P?
15. O que significa dizer que uma arquitetura é "escalável"?
16. Quais são as principais vantagens de uma arquitetura cliente-servidor?
17. Como um servidor web difere de um servidor de aplicação?
18. Qual é a vantagem de armazenar dados no servidor em uma arquitetura cliente-servidor?
19. Quais são as principais características de um servidor web?
20. Quais são as características de uma arquitetura P2P?

Perguntas e Respostas

1. O que é arquitetura de software?
   Resposta: A arquitetura de software é a estrutura organizacional de um sistema de software, incluindo componentes de software, as propriedades externamente visíveis desses componentes e as relações entre eles.
2. O que é coesão em arquitetura de software?
   Resposta: Coesão refere-se à medida em que os componentes de um módulo de software estão relacionados entre si. Em uma arquitetura altamente coesa, cada módulo é responsável por uma tarefa específica.
3. O que é acoplamento em arquitetura de software?
   Resposta: Acoplamento descreve o grau de interdependência entre os módulos de software. Em um sistema com baixo acoplamento, os módulos são independentes uns dos outros.



11225354498 - Maria Vitória Gonçalves Vieira

4. O que é arquitetura em camadas?
   Resposta: A arquitetura em camadas é um tipo de arquitetura de software que separa a funcionalidade do sistema em camadas, cada uma com funções específicas.
5. O que é arquitetura monolítica?
   Resposta: Arquitetura monolítica é uma arquitetura de software em que a aplicação é construída como uma única unidade indivisível.
6. O que é arquitetura em duas camadas?
   Resposta: A arquitetura em duas camadas divide a aplicação em duas partes principais: o cliente e o servidor.
7. O que é arquitetura em três camadas?
   Resposta: A arquitetura em três camadas divide a aplicação em três componentes principais: a interface do usuário ou camada de apresentação, a lógica de negócio e a camada de acesso a dados.
8. O que é a arquitetura MVC?
   Resposta: A arquitetura MVC é um padrão de design que divide a aplicação em três componentes interconectados: Modelo, Visão e Controlador.
9. O que é arquitetura distribuída?
   Resposta: A arquitetura distribuída é uma arquitetura em que os componentes do sistema são localizados em diferentes máquinas de rede.
10. O que é middleware?
    Resposta: Middleware é o software que atua como uma ponte entre diferentes aplicações, sistemas ou serviços, facilitando a comunicação e a gestão de dados.
11. O que é arquitetura de microsserviços?
    Resposta: A arquitetura de microsserviços é um estilo de arquitetura que estrutura a aplicação como uma coleção de serviços leves, independentemente implantáveis e escaláveis.
12. O que é arquitetura mainframe?
    Resposta: A arquitetura mainframe é uma arquitetura de computador centralizada em que uma grande máquina hospeda e executa todas as funções e serviços de processamento de dados.
13. O que é arquitetura P2P?
    Resposta: A arquitetura P2P (Peer-to-Peer) é uma arquitetura de rede descentralizada em que todas as partes (peers) têm as mesmas responsabilidades e capacidades, compartilhando diretamente com cada outra.
14. Qual é a diferença entre uma arquitetura cliente-servidor e uma arquitetura P2P?
    Resposta: Na arquitetura cliente-servidor, os clientes se comunicam com um servidor central, enquanto na arquitetura P2P, os nós participantes têm papéis semelhantes e se comunicam diretamente entre si.
15. O que significa dizer que uma arquitetura é "escalável"?
    Resposta: Uma arquitetura escalável é capaz de lidar com o aumento da demanda ou do

11225354498 - Maria Vitória Gonçalves Vieira

tamanho do sistema, permitindo que recursos sejam adicionados para acomodar o crescimento sem grandes impactos no desempenho.

16. Quais são as principais vantagens de uma arquitetura cliente-servidor?
    Resposta: Alguns benefícios da arquitetura cliente-servidor incluem divisão clara de responsabilidades, segurança aprimorada, controle centralizado, facilidade de manutenção e escalabilidade.
17. Como um servidor web difere de um servidor de aplicação?
    Resposta: Um servidor web é responsável por entregar conteúdo estático, enquanto um servidor de aplicação executa aplicativos complexos, fornecendo suporte a lógica de negócio, persistência de dados e outros recursos avançados.
18. Qual é a vantagem de armazenar dados no servidor em uma arquitetura cliente-servidor?
    Resposta: Armazenar dados no servidor melhora a segurança, permitindo a implementação de mecanismos de controle de acesso, criptografia e backup dos dados.
19. Quais são as principais características de um servidor web?
    Resposta: Fornecimento de conteúdo estático, suporte a protocolos web e tratamento de solicitações básicas são características de um servidor web.
20. Quais são as características de uma arquitetura P2P?
    Resposta: Descentralização, igualdade de papéis, escalabilidade, redundância, eficiência, autoorganização, privacidade e segurança são características de uma arquitetura P2P.

LISTA DE QUESTÕES ESTRATÉGICAS

1. **(UFG / SANEAGO – 2017)** O emprego de boas práticas de projeto (design) de software visa resultar em um código:

   a) altamente acoplado e altamente coeso.

   b) altamente acoplado e fracamente coeso.

   c) fracamente acoplado e altamente coeso.

   d) fracamente acoplado e fracamente coeso.

2. **(CESPE / TRE-BA – 2017)** Com referência às arquiteturas multicamadas de aplicações para o ambiente web, assinale a opção correta.

   a) Se, na camada de dados, for realizada uma alteração no banco de dados, o restante das camadas também será afetado.

11225354498 - Maria Vitória Gonçalves Vieira

b) O modelo de três camadas recebe essa denominação caso um sistema cliente-servidor seja desenvolvido mantendo-se a camada de negócio do lado do cliente e as camadas de apresentação e dados no lado do servidor.

c) Cada camada é normalmente mantida em um servidor específico para tornar-se o mais escalonável e independente possível em relação a outras camadas, estando entre as suas principais características o eficiente armazenamento e a reutilização de recursos.

d) O objetivo das arquiteturas multicamadas consiste na junção de responsabilidades entre os componentes das aplicações web, de modo a atender aos requisitos funcionais e não funcionais esperados pela aplicação.

e) Na arquitetura de duas camadas — apresentação e armazenamento —, o computador que contiver a base de dados terá de ficar junto com os computadores que executarem as aplicações.

3. **(IBFC / EBSERH – 2017)** O modelo de três camadas físicas (3-tier), especificado nas alternativas, divide um aplicativo de modo que a lógica de negócio resida no meio das três camadas, foi adaptado como uma arquitetura para as aplicações Web em todas as linguagens de programação maiores. Muitos frameworks de aplicação comerciais e não comerciais foram criados tendo como base a arquitetura:

   a) MVC (Model-View-Controller)

   b) MDB (Model-Data-Business)

   c) UDC (User-Data-Controller)

   d) MDC (Model-Data-Controller)

   e) UVB (User-View-Business).

4. **(FCC / TCM-GO – 2015 – Adaptada)** Quanto à Arquitetura em 3 Camadas, é necessário um arranjo que possibilite a reutilização do código e facilite sua manutenção e seu aperfeiçoamento. Deve- se separar Apresentação, Regra de Negócio e Acesso a Dados. Busca-se a decomposição de funcionalidades de forma a permitir aos desenvolvedores concentrarem-se em diferentes partes da aplicação durante a implementação.



11225354498 - Maria Vitória Gonçalves Vieira

5. **(FCC / CNMP – 2015)** Há algumas variantes possíveis de arquitetura a serem utilizadas em um sistema de bancos de dados. Sobre essas variantes, é correto afirmar que:

a) na arquitetura de 3 camadas, não há uma camada específica para a aplicação.

b) a camada de apresentação da arquitetura de 2 camadas situa-se, usualmente, no servidor de banco de dados.

c) na arquitetura de 3 camadas, a camada de servidor de banco de dados é denominada cliente.

d) a arquitetura de 3 camadas é composta pelas camadas cliente, aplicação e servidor de banco de dados.

e) na arquitetura de 2 camadas não há necessidade de uso de um sistema gerenciador de bancos de dados.

6. **(VUNESP - 2023 - TCM-SP - Auxiliar Técnico de Controle Externo - Técnico de Informática)**

A respeito do conceito de arquitetura de software em múltiplas camadas (multitier architecture), é correto afirmar que

A) as camadas de interface de usuário e lógica de negócio possuem separação conceitual e independência de implementação, mas suas implementações precisam ser executadas na mesma máquina. A camada de dados pode ser executada em outra máquina.

B) as camadas de lógica de negócio e de dados possuem separação conceitual e independência de implementação, mas suas implementações precisam ser executadas na mesma máquina. A camada de interface de usuário pode ser executada em outra máquina.

C) as implementações das diferentes camadas podem ser executadas em diferentes máquinas, podendo a aplicação estar dividida em mais de 3 camadas.

D) as implementações das diferentes camadas podem ser executadas em diferentes máquinas, mas a arquitetura possibilita, no máximo, 3 camadas.

E) todas as camadas possuem separação conceitual e independência de implementação entre si, e suas implementações precisam ser executadas na mesma máquina.

7. (VUNESP - 2020 - Prefeitura de Ilhabela - SP - Analista - Tecnologia da Informação e Comunicação)

A respeito da arquitetura cliente-servidor, utilizada por muitas aplicações, é correto afirmar:

A) O programa servidor da aplicação precisa estar acessível via Internet, por meio de um endereço IP público e com a porta de acesso aberta nos firewalls aplicáveis

B) Um cliente pode ser acessado por um ou vários servidores, os quais fazem requisições de serviços ou funcionalidades aos primeiros.

C) A arquitetura P2P (peer-to-peer) é um sinônimo para a arquitetura cliente-servidor.

D) É uma arquitetura imune a ataques do tipo DDoS (Distributed Denial-of-Service) e, por isso, é bastante usada em aplicações que funcionam pela Internet.

E) Pode ser utilizada localmente, ou seja, o programa servidor e o programa cliente podem ser executados na mesma máquina.

8. (VUNESP - 2020 - FITO - Técnico em Gestão - Informática)

Uma certa arquitetura de software de três camadas (three-tier) é composta pela camada de apresentação (presentation tier), camada de aplicação com lógica negócio (application/business tier) e camada de dados (data tier). Cada camada é implementada por um módulo independente do outro, podendo esses módulos ser executados em computadores diferentes, comunicando-se por TCP/IP. No contexto dessa arquitetura e considerando os princípios desse padrão arquitetural, é correto afirmar que a camada de

A) aplicação funciona como um servidor para a camada de apresentação, e como um cliente para a camada de dados.

11225354498 - Maria Vitória Gonçalves Vieira

B) apresentação acessa diretamente a camada de dados, e a camada de aplicação faz o monitoramento dessa comunicação.

C) dados tem por finalidade apresentar dados da aplicação na interface do usuário, tal como exibir produtos em uma aplicação de comércio eletrônico.

D) aplicação utiliza diretamente um sistema gerenciador de banco de dados relacional (SGBDR) para manter os dados persistentes da aplicação.

E) apresentação é implementada por um navegador de Internet, razão pela qual essa arquitetura é restrita a aplicações web.

9. (VUNESP - 2019 - Prefeitura de Itapevi - SP - Analista em Tecnologia da Informação e Comunicação)

No modelo de arquitetura Cliente-Servidor

A) um cliente não se comunica com outro cliente.

B) a função do servidor é receber as solicitações de serviços dos clientes receptores de serviços, e conectá-los aos clientes provedores de cada serviço requisitado.

C) caso o servidor não esteja em funcionamento, os clientes poderão solicitar serviços de outros clientes.

D) ocorre a comunicação direta entre os clientes, sem que haja uma separação fixa entre o cliente e o servidor.

E) um cliente envia uma solicitação de serviço pela rede, e outro cliente que pode atender executa o serviço, sob a supervisão do servidor.

10. (VUNESP - 2019 - UFABC - Técnico de Tecnologia da Informação)

Grande parte dos serviços da Internet utiliza a arquitetura Cliente/Servidor, pois apresenta, como vantagem,

A) a distribuição de solicitações nos Servidores, o que evita a sobrecarga.

B) a possibilidade de oferecer e disponibilizar serviços de forma espontânea

C) os dados serem armazenados no Servidor, o que melhora a segurança.

11225354498 - Maria Vitória Gonçalves Vieira

D) ser imune a falhas no Servidor, uma vez que há redundância de serviços.

E) utilizar um sistema operacional especializado, o que facilita a instalação.

11. (CESGRANRIO / CEFET-RJ – 2014) No contexto da Arquitetura de Sistemas, o MVC (model – view – controller) é um estilo arquitetural:

a) interativo

b) estrutural

c) distribuído

d) adaptável

e) monolítico

Gabaritos

1. C
2. Certo
3. A
4. C
5. D
6. C
7. E
8. A
9. A
10. C
11. A

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.